

Presentation 2 Documentation

• Links to presentation(s) and code(s) on GitHub:

- [Presentation Link](#) (Slides) \ \ [Presentation Link \(GitHub\)](#)
- Dashboard: [Power BI Sharing Link](#)
- [Data Prep Code Link](#) \ \ [Hierarchical Filtering Code Link](#)

• What did you do?

For this presentation, I updated my visuals from Presentation 1 on the Time Trends Dashboard and came up with new insights.

I started by moving all of the transformations I previously completed in Power BI over to Python, as requested by Prof. Krishna and Cynthia. I used the import code provided by Prof. Krishna for the CAR data and then made the following transformations in Python:

1. Grouped by Contact Session ID to ensure each call corresponds to a single session and prevent double-counting of calls.
2. Aggregated session-level metrics:
 - Call_Start_Time: timestamp of the first activity in the session
 - Starting_Hour: hour of day when the call started
 - Count: number of distinct activities per session
 - Call_Duration: difference (in minutes) between the first and last activity
 - DayOfWeekNum: numeric representation of the weekday (Monday = 1, Sunday = 7)
 - Call_Start_Date: Just the date (no timestamp) for the first activity of the session

I previously expanded the data in Power BI using the “All Rows” feature for filtering purposes.

Python does not allow for this feature, so I instead created a link between the transformed dataset and the original compiled dataset in Power BI via the Contact Session ID column. Then, I set the Cross-Filter direction of that relationship to “Both” to enable filtering. This allows me to

reference details about each call (such as Activity, EP, Queue, Agent, etc.) on an activity level, while still having only one row per call.

I then completed the following actions in Power BI:

3. Created Visualizations:

- Type: Bar Chart

X-axis: Starting Hour

Y-axis: Average Calls per Day → To build measure:

```
Avg Calls per Day =  
VAR TotalCalls = COUNTROWS('combined_calls_transformed_simple')  
VAR TotalDays =  
    DISTINCTCOUNT('combined_calls_transformed_simple'[Call_Start_Date])  
RETURN  
    DIVIDE(TotalCalls, TotalDays)
```

Legend: DayofWeekNum

- Type: Line Graph

X-axis: Starting Hour

Y-axis: Average of Call Duration per Hour → To build measure:

```
Average Call Duration per Starting Hour =  
AVERAGEX(  
    VALUES('combined_calls_transformed_simple'[Starting_Hour]),  
    CALCULATE(AVERAGE('combined_calls_transformed_simple'[Call_Duration]))  
)
```

Legend: DayofWeekNum

4. Added Hierarchical Filtering:

I used the Visio Map resource to categorize the unique values in the *Activity Name* column of the combined CAR data into a hierarchy by detailing each value with a tier and the tiers that precede it. I then created a dataframe which contained each tier as a separate column and filled in the hierarchy for each activity name, allowing for hierarchical filtering and analysis in Power BI without duplication or blank subfilters. I uploaded this file into Power BI and linked it with the original compiled CAR data via the Activity Name column. Again, I set the Cross-Filter direction of that relationship to “Both” to enable

filtering. I then set up the filtering using a list slicer in Power BI, as seen in the YouTube video provided by Prof. Krishna.

- **How does it help the project?**

This analysis of peak call time trends with various filters can help the team uncover operational inefficiencies and develop strategies to improve overall intake performance, as seen in my presentation. Based on my analysis, call volumes and durations generally peak in the morning and early in the week across most menus. However, certain categories—such as the **Seniors** menu and its submenus—show significantly longer average call durations, indicating more complex interactions that may require additional training or specialized support. Additionally, language-specific transfers like **StaffDirectorySpanishTransfer** peak later in the day compared to English transfers, highlighting opportunities to better align staffing schedules and expertise with caller behavior patterns.

- **Issues Resolved:**

- I was having trouble simultaneously grouping the data and retaining all the necessary activity level information in the CAR data. To solve this problem, I linked call-level and activity-level versions of the data in Power BI and enabled cross-filtering.
- I couldn't figure out how to create hierarchical tiers from just one row (Activity Name) in the data. To resolve this issue, I created a separate file with activity tiers and linked it to the CAR data to enable filtering. I then used a list slicer in Power BI and used each tier as a level for the hierarchical filter. I also noticed that a lot of smaller (less frequently visited menus) were populating the top of the filter because they did not belong to other larger menus. To solve this, I remade the activity tier file with a miscellaneous tier so these irrelevant menus could still be accessed without crowding the filter box.

- **Next steps**

For the next presentation, I plan to:

- Create another hierarchical filter for the “Queue Name” column in the CAR data for my dashboard, using the same process. From there, I will analyze trends in call volume and duration by time and weekday for different Queues and develop targeted recommendations.

- **References (Mention if you built up on someone else’s work)**

Code import files from Prof. Krishna

Presentation 1 Documentation

• Links to presentation(s) and code(s) on GitHub:

- [Presentation Link](#)
 - Note that if the Power BI embedded in the slides fails, static visuals can be found at the bottom of the slide deck
- All coding was done in Power BI: see below // [Power BI Sharing Link](#)

• What did you do?

I created visuals for the Time Trends Dashboard. I began by compiling all CAR data into a CSV file using the import code provided by Prof. Krishna. I then loaded the file into Power BI for analysis. I then performed the following transformations on the dataset *within Power BI*:

I transformed the data using the *Group By* function to ensure each *Contact Session ID* corresponded to only one row—preventing double counting caused by multiple actions per session. I used the *Advanced Group By* option to create multiple aggregations that support deeper analysis and visualization. Specifically, I created three new columns:

- Call Start Time — *Operation*: Minimum of *Activity Start Timestamp*
Purpose: Shows timestamp of the first row in a session to identify when the call began.
- Starting Hour — *Operation*: Minimum of *Hour*
Purpose: Records the hour of day when the session started.
- All Rows — *Operation*: All Rows
Purpose: Retains information from each session in nested tables for detailed reference.

I also *expanded* the other columns from this one to access nested information.

I then used those columns to create the following columns:

- Count - *Operation*: Row/activity count per call with distinct activity timestamp
Code: `List.Count(List.Distinct([All Rows][Activity Start Timestamp]))`
Purpose: See how many different menus/actions a user goes through in each call
- Call Duration - *Operation*: Subtract the minimum timestamp of a call from the maximum

Code: Duration.TotalMinutes(
 List.Max([All Rows][Activity Start Timestamp]) -
 List.Min([All Rows][Activity Start Timestamp])
)

Purpose: Determine how long each call lasted in minutes

- **DayOfWeekNum - Operation:** Extract weekday from Call Start Time Column as number (Monday = 1 and Sunday = 7)

Code: Date.DayOfWeek([Call Start Time], Day.Monday) + 1

After these transformations, I created the following visualizations:

- **Type:** Bar Chart
X-axis: Starting Hour
Y-axis: Contact Session
Legend: DayofWeekNum
- **Type:** Line Graph
X-axis: Starting Hour
Y-axis: Average of Activity Count per Hour → **Code for Measure:**
 Average Count per Starting Hour =
 AVERAGEX(
 KEEPFILTERS(VALUES('combined_calls'[Starting Hour])),
 CALCULATE(AVERAGE('combined_calls'[Count])))
- **Type:** Line Graph
X-axis: Starting Hour
Y-axis: Average of Call Duration per Hour → **Code for Measure:**
 Count of Call Duration average per Starting Hour =
 AVERAGEX(
 KEEPFILTERS(VALUES('combined_calls'[Starting Hour])),
 CALCULATE(COUNTA('combined_calls'[Call Duration])))

*On each visual, you can filter by: Activity Name, Agent Name, EP Name, Flow Name, Queue Name, Termination Reason, and both axes and legend.

• How does it help the project?

This analysis of peak call time trends with various filters can help the team uncover operational inefficiencies and develop strategies to improve overall intake performance, as seen in my presentation. Based on my analysis, changing the messaging for times users are encouraged to call could increase the quality of service for users and allow agents to reach more people overall. Additionally, developing a system for tracking call outcome/success will enable deeper analysis of call effectiveness, help identify when and why shorter calls are less successful, and guide future staffing and training decisions.

• Issues Resolved After Feedback:

- Activities with the exact same timestamp were counted as one and activities with different timestamps were counted as distinct.
- All 3 plots pull from the CAR data to ensure consistency.
- Conclusions about peak call times were adjusted to accurately reflect the dashboard.
- Recommendations were expanded to include all 3 plots and made more precise/actionable.

• Next steps

For the next presentation, I plan to:

- *Dive deeper into the EP/Activity/Flow/Queue filters* to clearly display trends in peak call times by different groups or legal issues
 - Use heatmaps or line charts to highlight patterns across hours and days.
- *Analyze average activities per call and call durations by weekday* in addition to time of day to discover new trends in calling behavior
- *Determine a metric/indicator for call success* which I observe according to time trends

• References (Mention if you built up on someone else's work)

Code import files from Prof. Krishna